▨ SyntaxFlow — Next-Generation Text-to-Diagram Engine

WASM-Driven · ELK Layouts · Orthogonal Routing · Highly Extensible

▨ Overview

SyntaxFlow is a highly sophisticated, deterministic text-to-diagram rendering engine.

Designed to solve the scaling and flexibility limitations of legacy tools like Mermaid.js, SyntaxFlow treats diagram generation as a strict compiler and graph-mathematics problem. It utilizes a lightning-fast Go-based WebAssembly parser, mathematically rigorous edge routing, and a decoupled component architecture to handle complex system diagrams without layout degradation.

▨ Why SyntaxFlow?

Legacy diagramming tools often suffer from rigid styling, unpredictable layouts with cyclic graphs, and overlapping edge routing.

SyntaxFlow solves this through:

- ▨ **WASM-Powered Parsing:** A custom Go parser compiled via TinyGo for near-zero latency text processing and a sub-100KB binary footprint.
- ▨ **Cycle Immunity:** Automated DFS back-edge removal to process complex microservice loops as Directed Acyclic Graphs (DAGs).
- ▨ *A Orthogonal Routing:** Intelligent, grid-based pathfinding to ensure lines never clip through nodes.
- ▨ **Dynamic Component Registry:** Define or import custom SVG nodes dynamically—no hardcoded shape limitations.
- ▨ **First-Class Developer UX:** Out-of-the-box support for high-contrast, neobrutalist, and terminal-based aesthetics.

▨ Core Engine Architecture

```
graph TD
    Text["Raw DSL Text"] --> WASM["Go / TinyGo Parser (WASM)"]
    WASM --> AST["Serialized JSON AST"]

    AST --> Bridge["TypeScript Engine Bridge"]
    Bridge --> CycleBreaker["DFS Cycle Breaker (State Tree)"]
```

```
    CycleBreaker --> ELK["ELK Layout Engine (Web Worker)"]
    ELK --> Router["A* Orthogonal Edge Router"]

    Router --> Registry["Component Registry (Shape DB)"]
    Registry --> DOM["Virtual DOM / SVG Renderer"]

    DOM --> Output["Final SVG / Interactive Canvas"]
```

⚙ The Rendering Pipeline

SyntaxFlow splits diagram generation into three highly specialized, decoupled pipelines.

1. The Compiler Pipeline (Go / WASM)

Instead of brittle Regex in JavaScript, SyntaxFlow leverages the Go standard library for robust string manipulation.

- **Lexing & Parsing:** A Go-based recursive descent parser converts the domain-specific language (DSL) into a strongly typed Abstract Syntax Tree (AST).
- **WASM Bridge:** Compiled using TinyGo for an ultra-lightweight footprint, the parser serializes the AST into JSON and passes it instantly to the web runtime.

2. The Layout & Routing Pipeline (The Math)

This is the core differentiator of the engine, treating space as a calculable grid.

- **Cycle Management:** Runs Depth-First Search (DFS) to identify and temporarily detach back-edges, preventing layout engine deadlocks.
- **Coordinate Mapping:** Utilizes Eclipse Layout Kernel (elkjs) in a Web Worker for highly optimized, layered node placement without blocking the UI thread.
- **Edge Routing:** Deploys an A* (A-Star) search algorithm to calculate the absolute shortest path for connector lines while treating nodes as collision obstacles.

3. The Custom Shape Registry

SyntaxFlow introduces a file-system-like approach to UI components.

- Acts as a key-value store mapping syntax (e.g., `[Database]`) to specific SVG generation functions.
- Allows developers to inject external JSON/SVG schemas at runtime to completely override default shapes or introduce custom brand assets.

▨ Playground Interface

Status: ▨ In Progress

To showcase the engine, the official playground features a distinct ctOS-inspired, neobrutalist terminal aesthetic.

Playground Features:

- Live-updating split-pane rendering.

- Direct SVG code injection via the terminal interface to test custom shapes in real-time.
- Raw telemetry output (WASM execution time, AST node count, layout iterations).

▨ Tech Stack

Core Engine

- **Parser:** Go / TinyGo (Compiled to WebAssembly)
- **Layout Engine:** elkjs (running in Web Workers)
- **Routing:** Custom A* Grid Algorithm
- **Rendering:** React / Virtual DOM (SVG Output)

Web Playground

- **Framework:** Next.js
- **Styling:** Tailwind CSS (Terminal/Neobrutalist preset)
- **State Management:** Zustand

▨ Roadmap

Phase 1 — Core Math & Parsing

- [ ] Build the Go Lexer and define the base AST struct definitions.
- [ ] Compile parser via TinyGo and establish the syscall/js WASM bridge.
- [ ] Implement the DFS cycle-breaking algorithm in TypeScript.
- [ ] Integrate elkjs for base node coordinate mapping.

Phase 2 — Visuals & Extensibility

- [ ] Build the Component Registry key-value store.
- [ ] Implement the A* Orthogonal Router.
- [ ] Develop the neobrutalist web playground.

Phase 3 — Advanced Features

- [ ] Live data-binding capabilities (Prometheus/Grafana telemetry integration on nodes).
- [ ] Language Server Protocol (LSP) for VS Code autocomplete.